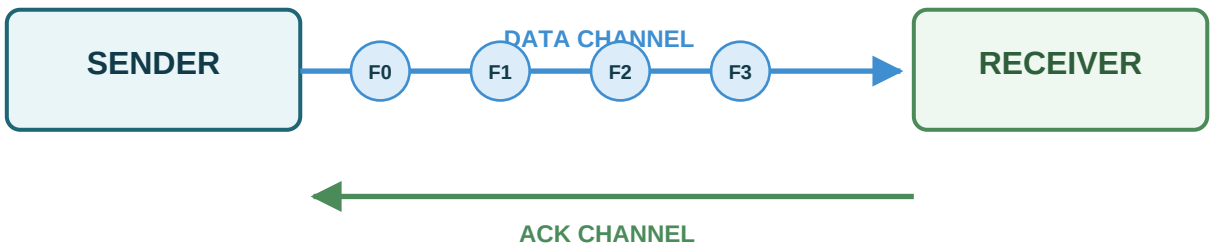
DATA COMMUNICATION AND NETWORKING

SLIDING WINDOW PROTOCOL SIMULATOR

Go-Back-N & Selective Repeat

Mini Project / Innovative Assignment
Course: Data Communication and Networking (3CS214CC26)
Academic Year: Odd Semester 2026–27
Student Name: Jaspreet Uppal, Hir Patel
Enrollment / Roll No.: 24BCS069, 24BCS073



Frames move through a sender window; ACKs return from the receiver.

A graphical simulation tool demonstrating sliding-window flow control, acknowledgements, packet loss, ACK loss, timeout and retransmission behaviour.

Declaration

This report documents a student-developed simulation tool for demonstrating the Go-Back-N and Selective Repeat sliding-window protocols. The protocol behaviour, simulation events, retransmission logic and graphical interface are implemented as part of the project source code.

The report is intended to accompany the submitted Python source files and to support demonstration of the networking concepts implemented in the simulator.

Item	Details
Project type	DCN Innovative Assignment / Mini Project
Implementation language	Python
GUI framework	Tkinter
Source files	main.py and protocol.py
Protocols	Go-Back-N and Selective Repeat
Simulation model	Sender–channel–receiver with ACK feedback

Table of Contents

No.	Section
1	Introduction
2	Problem Statement and Objectives
3	Background: Sliding Window Protocol
4	Go-Back-N Protocol
5	Selective Repeat Protocol
6	System Design and Architecture
7	Implementation
8	Graphical User Interface
9	Simulation Workflow
10	Testing and Demonstration Cases
11	Comparison of the Two Protocols
12	Limitations and Scope
13	Conclusion
14	Source Code Structure

Submission note: The assignment brief specifies that the submission should include a Word/PDF description of the protocol or network together with the applicable source code. This report is prepared as the documentation component; the two Python files should be submitted alongside it.

1. Introduction

A sliding-window protocol is a data-link-layer mechanism used to control how multiple frames can be sent before acknowledgements are received. Instead of waiting for an acknowledgement after every individual frame, the sender can have several outstanding frames within a defined window. As acknowledgements arrive, the window advances and additional frames can be transmitted.

This project implements a graphical simulator for two sliding-window ARQ approaches: **Go-Back-N (GBN)** and **Selective Repeat (SR)**. The simulator allows the user to configure the number of frames, window size, packet-loss probability, ACK-loss probability and animation speed. It then displays the protocol operation event by event.

The primary purpose is educational demonstration. The GUI makes protocol events visible so that the user can observe successful delivery, packet loss, acknowledgement loss, timeout and retransmission behaviour.

2. Problem Statement and Objectives

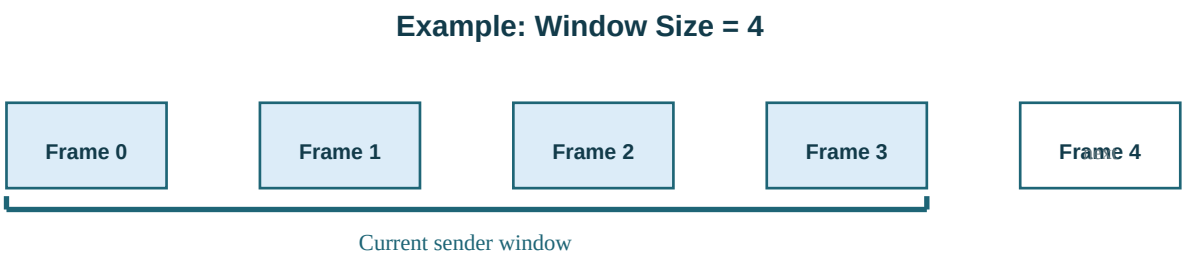
Problem Statement: Develop a simulation tool that demonstrates the operation of a sliding-window protocol using Go-Back-N and Selective Repeat, with a user interface suitable for explaining the concept and observing protocol events.

Objectives

- Implement Go-Back-N sliding-window behaviour from scratch in Python.
- Implement Selective Repeat-style selective retransmission behaviour.
- Represent a sender, receiver, data channel and ACK channel graphically.
- Allow configurable frame count and sender window size.
- Simulate packet loss and ACK loss.
- Show timeout and retransmission events in an event log.
- Provide START, STOP and RESET controls for demonstration.
- Display summary information such as frames delivered, total transmissions and retransmissions.

3. Background: Sliding Window Protocol

In a sliding-window protocol, the sender maintains a range of sequence numbers that are currently permitted to be outstanding. The size of this range is the **window size**. When a frame is successfully acknowledged, the sender can move the lower edge of the window forward and send another frame.



The project models the communication path as two logical channels. The forward channel carries data frames from sender to receiver, while the reverse channel carries acknowledgements. Packet loss and ACK loss are simulated independently to demonstrate why retransmission mechanisms are required.

Key Terms

Term	Meaning in the simulator
Frame	A unit of data transmitted from the sender to the receiver.
Window	The group of frames currently allowed to be outstanding.
ACK	Acknowledgement indicating successful reception of a frame.
Packet loss	Simulated loss of a data frame on the forward channel.
ACK loss	Simulated loss of an acknowledgement on the return channel.
Timeout	The sender's indication that an expected acknowledgement did not arrive.
Retransmission	Sending a frame again after loss or timeout.

4. Go-Back-N Protocol

In the Go-Back-N approach implemented in this project, the sender can transmit several frames within its current window. When a frame or its ACK is lost, the simulator generates a timeout event and the sender returns to the first unacknowledged frame. Frames from that point are then transmitted again.

Typical Sequence

- The sender determines the current window of frames.
- Frames in the window are transmitted one by one.
- The simulator checks whether a data packet is lost.
- If the frame arrives, the receiver generates an ACK event.
- The simulator independently checks whether the ACK is lost.
- If an ACK is received, the sender advances its base and the window moves.
- If a packet or ACK is lost, a timeout is recorded and retransmission begins from the unacknowledged frame.

```
# Simplified Go-Back-N logic used by the simulator

while base < total_frames:
send frames inside current window

if packet is lost:
timeout
next_frame = base
retransmit from base

elif ACK is lost:
timeout
next_frame = base
retransmit from base

else:
receive ACK
advance base
slide window
```

Example Event Sequence

Step	Event	Meaning
1	Sender Window: [0, 1, 2, 3]	Four frames are currently permitted.
2	Sending Frame 0	Frame 0 enters the data channel.
3	Frame 0 received	Receiver successfully receives the frame.
4	ACK 0 received	Sender can advance its window.
5	Frame 2 lost	A data packet is lost.
6	Timeout for Frame 2	Sender detects missing acknowledgement progress.
7	Retransmission	Transmission resumes from the unacknowledged frame.

5. Selective Repeat Protocol

Selective Repeat differs from Go-Back-N in its retransmission behaviour. When a particular frame is lost or its acknowledgement is lost, the simulator records a timeout for that frame and retransmits that frame rather than restarting transmission from the beginning of the outstanding set.

Typical Sequence

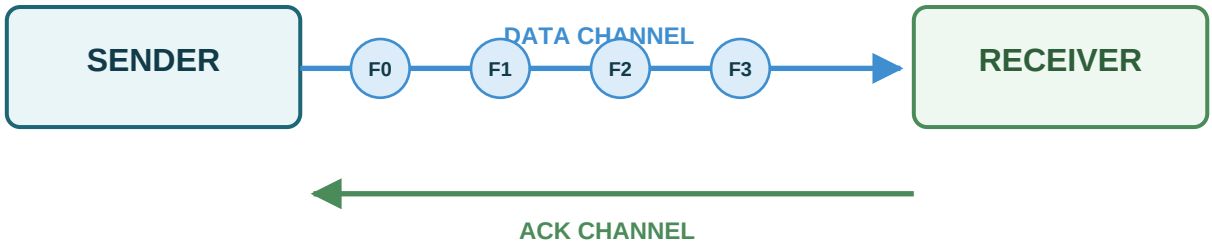
- The sender determines the current window.
- Each unacknowledged frame in the window is transmitted.
- A lost frame is marked as lost and receives a timeout event.
- A successfully received frame may generate an ACK.
- An ACK loss causes the corresponding frame to remain unacknowledged.
- The simulator later retransmits only the affected frame.
- Acknowledged frames are retained as acknowledged and are not unnecessarily retransmitted.

```
# Simplified Selective Repeat logic used by the simulator

for frame in current_window:
if frame is already acknowledged:
skip it

transmit frame

if packet is lost:
timeout for this frame
continue

if ACK is lost:
timeout for this frame
continue

mark frame as acknowledged
slide window when possible
```

6. System Design and Architecture

The project is divided into two Python source files. This separation keeps the protocol simulation independent from the graphical interface.

File	Responsibility
main.py	Creates the Tkinter interface, validates user input, controls animation, displays the network visualization and event log, and handles START/STOP/RESET.
protocol.py	Contains the simulation logic for Go-Back-N and Selective Repeat and returns event records plus summary statistics.



Frames move through a sender window; ACKs return from the receiver.

Data Flow

Stage	Operation
1. Input	User selects protocol and enters simulation parameters.
2. Simulation	protocol.py generates the sequence of protocol events.
3. Animation	main.py presents those events one at a time using Tkinter scheduling.
4. Visualization	The current frame and status are displayed in the network panel.
5. Logging	Every important protocol event is written to the event log.
6. Summary	After completion, delivered frames, total transmissions and retransmissions are shown.

7. Implementation

Input Parameters

Parameter	Default	Purpose
Protocol	Go-Back-N	Selects the ARQ mechanism.
Number of Frames	10	Total number of frames to deliver.
Window Size	4	Maximum number of frames considered in the current window.
Packet Loss	20%	Probability used to simulate data-frame loss.
ACK Loss	10%	Probability used to simulate ACK loss.

Parameter	Default	Purpose
Animation Speed	Normal	Controls the delay between displayed events.

Input Validation

The interface validates that the number of frames is positive, the window size is at least one and does not exceed the number of frames, and packet/ACK loss values remain within the supported range. The implementation avoids accepting 100% loss in the GUI because a simulation with every packet or ACK lost would not provide normal completion behaviour.

Event Representation

The protocol module represents each event using a small record containing the event message, an optional frame number and a status. Example statuses include TRANSMITTING, PACKET LOST, RECEIVED, ACK LOST, ACK RECEIVED and TIMEOUT. The GUI interprets these statuses to update the network visualization.

Animation and Control

The GUI uses Tkinter's scheduled callback mechanism to display one event at a time. The returned scheduling identifier is stored so that the animation can be cancelled when the user presses STOP or RESET. This prevents a previous simulation from continuing after the interface has been reset.

8. Graphical User Interface

The simulator is designed as a compact desktop application. The main screen contains the project title, configuration controls, network visualization, event log, statistics and simulation controls.

GUI Area	Function
Simulation Settings	Protocol, frame count, window size, packet loss, ACK loss and animation speed.
Network Visualization	Displays sender, receiver, DATA channel, ACK channel and current frame/status.
Simulation Event Log	Shows the sequence of protocol events in chronological order.
Statistics	Shows delivered frames, total transmissions and retransmissions after completion.
START	Begins a new simulation using the current settings.
STOP	Cancels the current animation and leaves the current display visible.
RESET	Cancels the current animation and restores default settings.

Demonstration Controls

- **START:** validates inputs, generates events and begins the animated display.
- **STOP:** cancels the pending animation callback and returns the interface to an editable state.
- **RESET:** cancels any pending animation, clears the event log and restores default values.
- **Animation Speed:** provides Slow, Normal and Fast modes so that the event sequence can be observed clearly.

9. Simulation Workflow

Step	Stage	Description
1	Configure	Select GBN or SR and set frame/window/loss parameters.
2	Start	Press START to create the simulation event sequence.
3	Transmit	Frames are shown moving through the DATA channel.
4	Receive	Successful frames produce receiver events.
5	ACK	Acknowledgements are shown returning through the ACK channel.
6	Failure	Packet loss or ACK loss produces a failure/timeout event.
7	Retransmit	The selected protocol determines which frames are retransmitted.
8	Complete	The simulator reports summary statistics.

10. Testing and Demonstration Cases

The following cases are suitable for a classroom demonstration. Because loss is simulated probabilistically, the exact event sequence can vary between runs.

Test	Settings	Expected observation
A. Normal transmission	Loss = 0%, ACK Loss = 0%	Frames should progress without loss/timeout events.
B. Packet-loss demonstration	Packet Loss = 20%, ACK Loss = 0%	Some runs show packet loss followed by timeout and retransmission.
C. ACK-loss demonstration	Packet Loss = 0%, ACK Loss = 20%	Some runs show ACK loss and subsequent retransmission.
D. Go-Back-N	Protocol = GBN	A timeout causes retransmission beginning from the unacknowledged frame.
E. Selective Repeat	Protocol = SR	A timeout is associated with the affected frame and selective retransmission is shown.
F. Stop/Reset	Any running case	STOP halts animation; RESET returns the interface to defaults.

Suggested Live Demonstration

- First run Go-Back-N with packet loss and ACK loss set to 0% so the basic window movement is easy to explain.
- Then increase packet loss to demonstrate timeout and retransmission.
- Switch to Selective Repeat with the same frame/window settings and compare the retransmission event sequence.
- Finally demonstrate STOP during an active animation and RESET to show that the GUI controls work correctly.

11. Comparison of the Two Protocols

The simulator is intended to make the retransmission difference between the two approaches observable. The following comparison describes the implemented behaviour without assigning an overall ranking.

Feature	Go-Back-N	Selective Repeat
Window-based transmission	Yes	Yes
ACK handling	Cumulative ACK event is shown	Individual ACK event is shown
Packet-loss response	Retransmission begins from the first unacknowledged frame	Affected frame is retransmitted selectively
ACK-loss response	Timeout causes retransmission from the unacknowledged point	Timeout is associated with the affected frame
Event visualization	Yes	Yes
Loss simulation	Packet and ACK loss	Packet and ACK loss

12. Limitations and Scope

The application is a teaching simulator rather than a full network stack or a real network implementation. The following scope points should be understood when demonstrating the project:

- The communication channel is simulated inside the Python program; no physical network connection is created.
- Packet and ACK loss are generated probabilistically rather than measured from a real network.
- The GUI visualizes protocol events rather than transmitting actual Ethernet/IP packets.
- The Selective Repeat implementation focuses on selective retransmission behaviour; it is intentionally kept compact for demonstration.
- The simulator uses a safety limit on attempts so that unusually adverse loss conditions do not cause an endless demonstration.

13. Conclusion

The Sliding Window Protocol Simulator provides a graphical way to study Go-Back-N and Selective Repeat behaviour. It combines configurable transmission conditions with an event-by-event visualization of frames, acknowledgements, loss, timeout and retransmission.

The project demonstrates the central idea of sliding-window flow control and ARQ-based error recovery through an interactive interface. Separating the GUI in **main.py** from protocol logic in **protocol.py** also makes the implementation easier to understand and demonstrate.

For submission, the PDF report should be accompanied by the two Python source files and demonstrated using representative loss and no-loss test cases.

14. Source Code Structure

```
SlidingWindowSimulator/
■
■■■ main.py
■ ■■■ Tkinter window and widgets
■ ■■■ Input validation
■ ■■■ Network visualization
■ ■■■ Event log
■ ■■■ Animation
■ ■■■ START / STOP / RESET
```

```
■ ■■■ Simulation statistics
■
■■■ protocol.py
■■■ Event creation
■■■ Simulation dispatcher
■■■ Go-Back-N simulation
■■■ Selective Repeat simulation
■■■ Summary generation
```

Reference used for assignment scope: Nirma University, Data Communication and Networking (3CS214CC26), "Innovative Assignment", Odd Semester 2026-27. The brief lists sliding-window Go-Back-N / Selective Repeat under the protocol implementation option and requires a Word/PDF description together with the applicable source code.